



Joystick

R. Ferrero, P. Bernardi

Politecnico di Torino

Dipartimento di Automatica e Informatica (DAUIN)

Torino - Italy

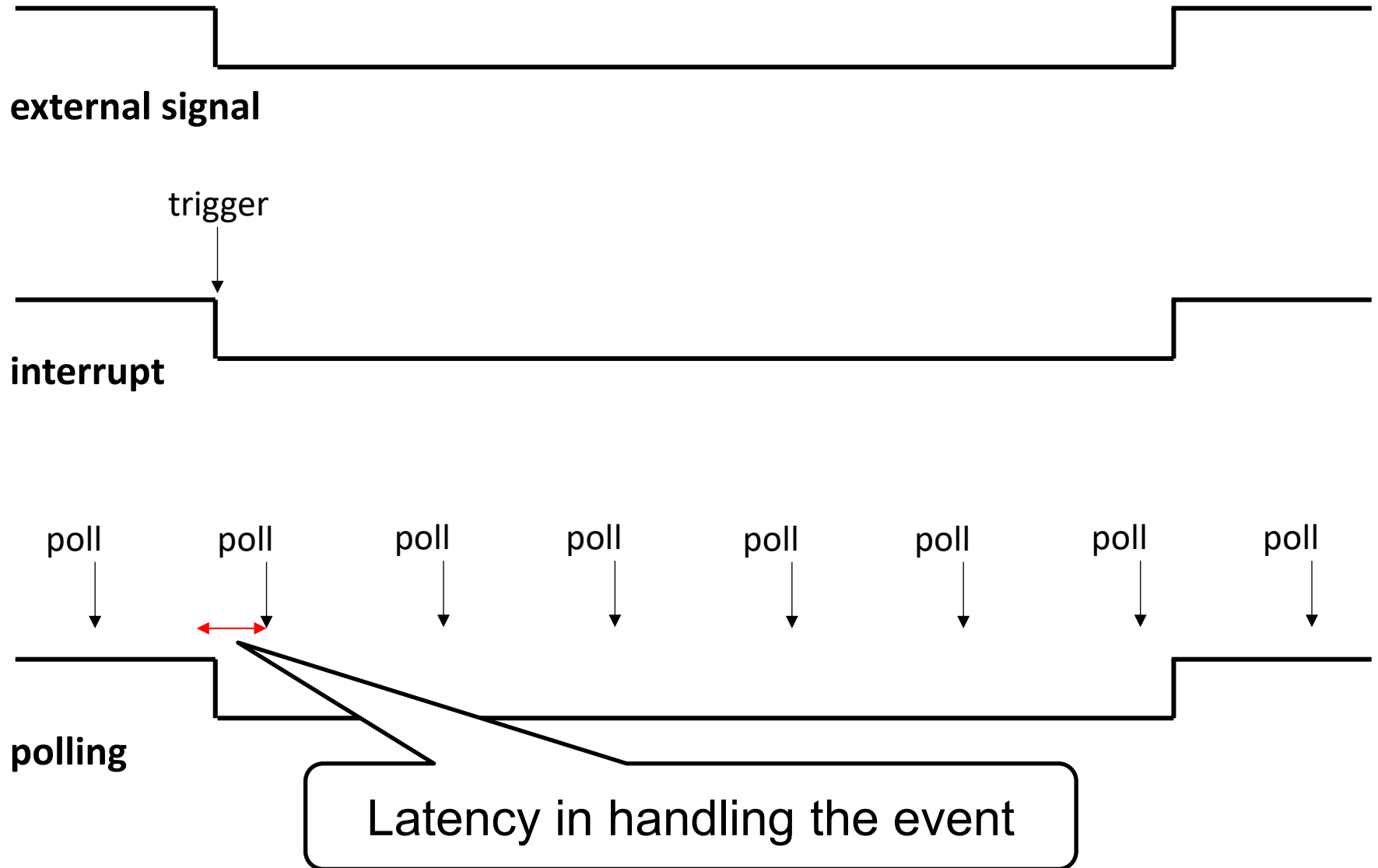
This work is licensed under the Creative Commons (CC BY-SA) License. To view a copy of this license, visit <http://creativecommons.org/licenses/by-sa/3.0/>



Interrupt vs Polling

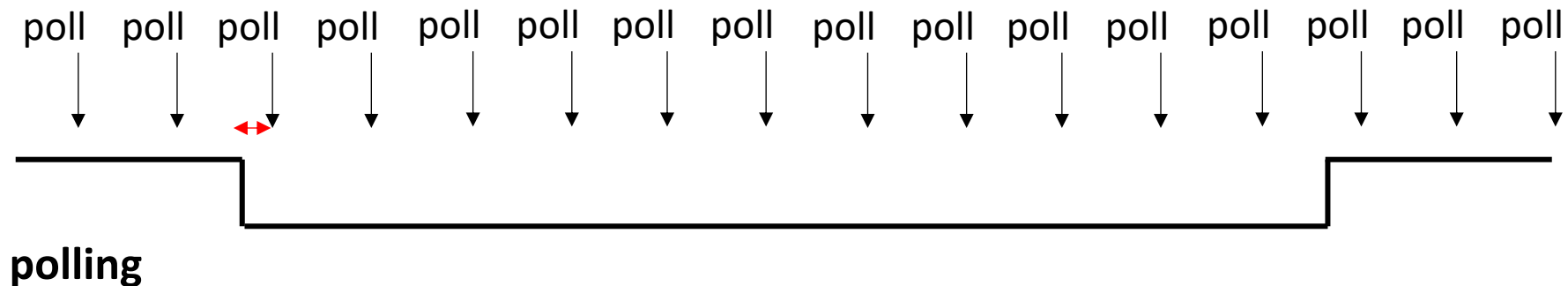
- Interrupt functionalities are preferred to be noticed about external events
 - time efficiency: the event is handled immediately without any latency (fundamental for safety)
 - power efficiency: the system sleeps between requests (fundamental for mobile devices)
- Not all events are triggering interrupts
 - Peripheral cores, inside the SoC, if not connected to the Interrupt Controller (quite unusual)
 - External devices connected to pins that cannot be configured as external interrupt sources (joystick).

Latency in polling



Latency mitigation

- Latency can be reduced by tighten the polling frequency
- Power efficiency is traded-off with latency
- The programmer needs to know about the characteristics of external input

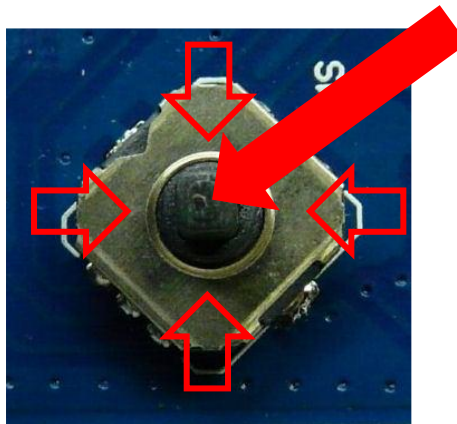


Polling implementation

- Software approach
 - `while(1)`
 - `poll(register);`
- Timer based approach
 - An interruption is triggered at regular intervals
 - The system sleeps while the timer is counting
- Low latency
- Power inefficient
- Trade-off between latency and power efficiency

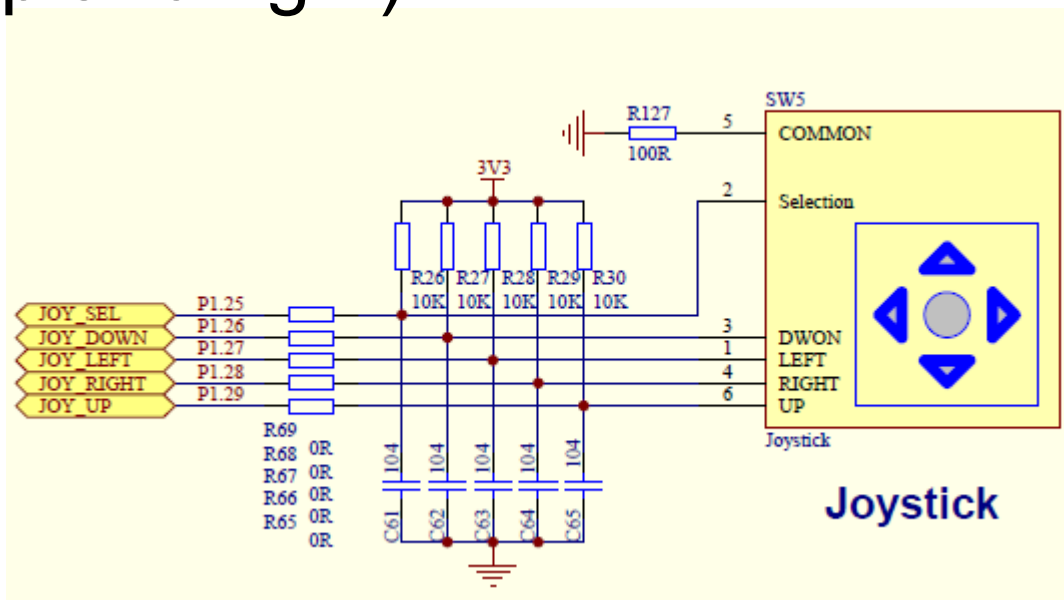
Case of study: the joystick

- The LandTiger LPC17XX board features a 5 way digital joystick (SW5).
- Each direction (up, down, left, right) and the Select function are connected to a dedicated digital input pin on the LPC1768.
 - The Select function is similar to a push button



Pin identification

- Input pins are active low when a key is pressed
- The input pins are hardware debounced.
- Multiple keys can be pressed at the same time (e.g. up and right).



Limitations

- The joystick is connect to pins without external interrupt capabilities

SDA1	P1.0/ENET_TXD0	95	P1.0	ENET TXD0
SCL1	P1.1/ENET_TXD1	94	P1.1	ENET TXD1
7	P1.4/ENET_TX_EN	93	P1.4	ENET_TX_EN
6	P1.8/ENET_CRS	92	P1.8	ENET CRS
/RD2/CAP2.0	P1.9/ENET_RXD0	91	P1.9	ENET_RXD0
TD2/CAP2.1	P1.10/ENET_RXD1	90	P1.10	ENET_RXD1
/SSEL1/MAT2.0	P1.14/ENET_RX_ER	89	P1.14	ENET_RX_ER
/SCK1/MAT2.1	P1.15/ENET_REF_CLK	88	P1.15	ENET_REF_CLK
MISO1/MAT2.2	P1.16/ENET_MDC	87	P1.16	ENET_MDC
/MOSI1/MAT2.3	P1.17/ENET_MDIO	86	P1.17	ENET_MDIO
2/MAT3.0	P1.18/USB_UP_LED/PWM1.1/CAP1.0	32	P1.18	USB_UP_LED
2/MAT3.1	P1.19/MC0A/USB_PPWR/CAP1.1	33	P1.19	USB_PPWR
0/SCK	P1.20/MCFB0/PWM1.2/SCK0	34	P1.20	SCK0
L0/SSEL	P1.21/MCABORT/PWM1.3/SSEL0	35	P1.21	SD_CS
00/MISO	P1.22/MC0B/USB_PWRD/MAT1.0	36	P1.22	USB_PWRD
SIO/MOSI	P1.23/MCFB1/PWM1.4/MISO0	37	P1.23	MISO0
1	P1.24/MCFB2/PWM1.5/MOSI0	38	P1.24	MOSI0
1	P1.25/MC1A/MAT1.1	39	P1.25	JOY_SEL
	P1.26/MC1B/PWM1.6/CAP0.0	40	P1.26	JOY_DOWN
	P1.27/CLKOUT/USB_OVRCR/CAP0.1	43	P1.27	JOY_LEFT
RX_CLK/CAP3.0	P1.28/MC2A1.0/MAT0.0	44	P1.28	JOY_RIGHT
RX_WS/CAP3.1	P1.29/MC2B/PCAP1.1/MAT0.1	45	P1.29	JOY_UP
RX_SDA/TXD3	P1.30/VBUS/AD0.4	21	P1.30	VBUS
UT/RXD3	P1.31/SCK1/AD0.5	20	P1.31	ADC_IN

Reading values from pins

- Setup the pins connected to the 5-way joystick actuators as
 - GPIO
 - in input direction
- Poll the GPIO register value
 - Retrieving a full port value
 - Then making the proper bits to selectively notice a change of status

Polling implementation

- The RIT (Repetitive Interrupt Timer) is used to implement the polling functionalities
- Every time (50ms) the RIT triggers an interrupt
 - This timing looks fine for interfacing human behavior (finger pressure)

Select function in RIT Handler

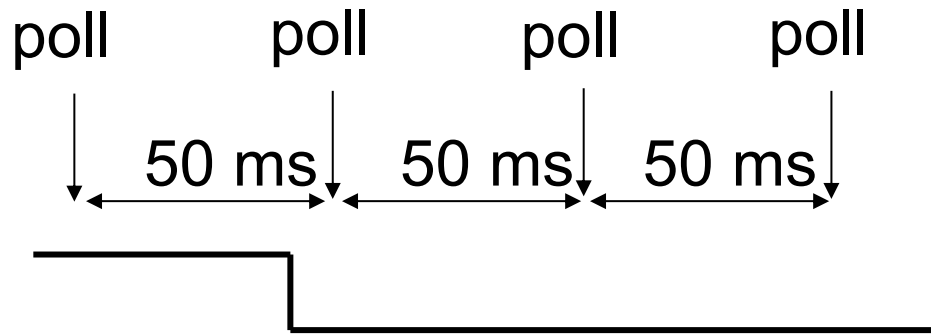
- The value of the port GPIO1 is read
- If the value of bit 25 is 0 then
 - If it is the first time: an action is performed
 - If it is a pressure repetition, no action is performed (push button functionality).

Select function in RIT Handler

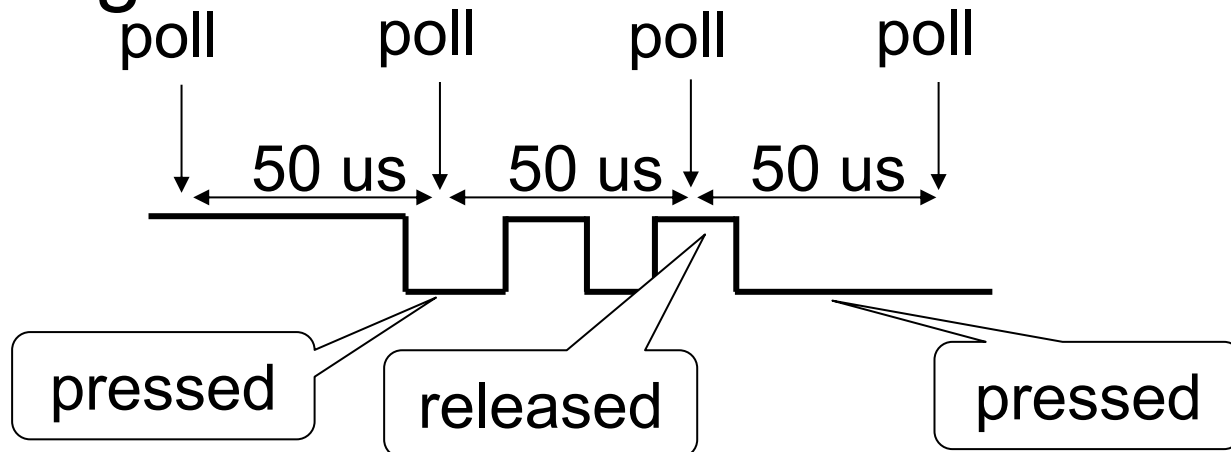
```
26 void RIT_IRQHandler (void)
27 {
28     static int select=0;
29
30     if((LPC_GPIO1->FIOPIN & (1<<25)) == 0){
31         /* Joytick Select pressed */
32         select++;
33         switch(select){
34             case 1:
35                 /* your action here */
36                 break;
37             default:
38                 break;
39         }
40     }
41     else{
42         select=0;
43     }
```

Bouncing and prolonged pressure

- Current scenario: hardware debounced



- Potential scenario: with faster polling and bouncing issues



Bouncing and prolonged pressure

```
26 void RIT_IRQHandler (void
27 {
28     static int select=0;
29
30     if((LPC_GPIO1->FIOPIN
31         /* Joystick Selection
32         select++;
33         switch(select){
34             case 1:
35                 /* your action here */
36                 break;
37             default:
38                 break;
39         }
40     }
41     else{
42         select=0;
43     }
```

This value may be changed to a multiple of the polling interval, depending on the bouncing timing characteristics

It may work in the potential scenario, but introduces a latency

Other cases can be included to manage special joystick functionalities, like prolonged pressure

Button and joystick with RIT only

- The `down` variable is used to synchronize the handlers
 - It is declared volatile to avoid optimization issues
- If the Ext Int handler was executed, `down` is different from 0
- Since the first read of button pins can be unreliable, the confirmation of the pressure is given after one polling cycle.

```

24 volatile int down=0;
25
26 void RIT_IRQHandler (void)
27 {
28     static int select=0;
29
30     if((LPC_GPIO1->FIOPIN & (1<<25)) == 0){ /* Joytick Select pre:
31         select++;
32         switch(select){
33             case 1:
34                 /* your action here */
35                 break;
36             default:
37                 break;
38         }
39     }
40     else
41         select=0;
42
43     if(down!=0){ /* button management */
44         if((LPC_GPIO2->FIOPIN & (1<<11)) == 0){ /* KEY1 pressed */
45             down++;
46             switch(down){
47                 case 2:
48                     /* your action here */
49                     break;
50                 default:
51                     break;
52             }
53         }
54         else { /* button released */
55             down=0;
56             NVIC_EnableIRQ(EINT1_IRQn); /* enable Button
57             LPC_PINCON->PINSEL4 |= (1 << 22); /* External inte:
58         }
59     }

```